ENGENHARIA DE *SOFTWARE*

Aline Zanin

Izabelly Soares de Morais

Revisão técnica:

Jeferson Faleiro Leon
Desenvolvimento de Sistemas
Especialista em Formação Pedagógica de Professores

COPIA NÃO AUTORIZADA É CRIME
ABDR
ASSOCIAÇÃO BRASILEIRA DE DIREITOS REPROGRÁFICOS
RESPEITE O DIREITO AUTORAL

M827e Morais, Izabelly Soares de.
Engenharia de software [recurso eletrônico] / Izabelly Soares de Morais, Aline Zanin ; revisão técnica : Jeferson Faleiro Leon. – Porto Alegre : SAGAH, 2017.

ISBN 978-85-9502-253-9

Engenharia. 2. Engenharia de software auxiliada por computador. I. Zanin, Aline. II. Título.

CDU 004.41

Catalogação na publicação: Karin Lorien Menoncin CRB-10/2147

Entender a fase de projeto (modelagem) de um sistema

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar os conceitos fundamentais de projeto.
- Reconhecer a importância das informações cobertas por diagramas.
- Diferenciar os diagramas da UML na modelagem de sistemas.

Introdução

Você sabia que, após o levantamento e a análise dos requisitos de software, você precisa fazer um projeto de como um sistema funcionará internamente? Neste projeto, é importante analisar questões como arquitetura do sistema, linguagens de programação que serão usadas, banco de dados, interface gráfica e outros elementos, além de gerar uma descrição computacional do que o software irá fazer, de forma que acompanhe os objetivos que foram obtidos na análise de requisitos.

Podemos dividir esta fase em duas partes: **projeto da arquitetura** (também chamado de projeto de alto nível) e **projeto detalhado** (também chamado de projeto de baixo nível).

Neste capítulo, você vai adquirir conhecimentos fundamentais sobre a fase de projeto (modelagem) de um sistema na engenharia de software. Você vai, ainda, estudar conceitos básicos sobre o que é a fase de projeto e as suas divisões e características.

Conceitos fundamentais de um projeto

Um projeto de software "[...] é uma descrição da estrutura do software a ser implementado, dos modelos e das estruturas de dados usados pelo sistema, das interfaces entre os componentes do sistema e, às vezes, dos algoritmos usados. Os projetistas não chegam a um projeto final imediatamente, mas o

desenvolvem de forma iterativa. Eles acrescentam formalidade e detalhes enquanto desenvolvem seu projeto por meio de revisões constantes para correção de projetos anteriores" (SOMMERVILLE, 2011, p. 25).

As fases que contemplam o projeto de um software, assim como as demais, são sequenciais e envolvem diversas vertentes do software. É importante ressaltar que o projeto é desenvolvido de acordo com as particularidades do sistema. Dentre elas, Sommerville (2011) destaca quatro atividades que podem ou não fazer parte do processo de projeto de sistemas de informação:

1. Projeto de arquitetura, no qual pode ser identificada a estrutura geral do sistema, os componentes principais (algumas vezes, chamados de subsistemas ou módulos), seus relacionamentos e como eles são distribuídos.
2. Projeto de interface, no qual se define as interfaces entre os componentes do sistema. Essa especificação de interface deve ser inequívoca. Com uma interface precisa, um componente pode ser usado de maneira com que outros componentes não saibam como ele é implementado.
3. Projeto de componente, no qual se toma cada componente do sistema e se projeta seu funcionamento. Pode-se tratar de uma simples declaração da funcionalidade que se espera implementar, com o projeto específico para cada programador. Este modelo de projeto pode ser usado para gerar automaticamente uma implementação.
4. Projeto de banco de dados, no qual se projeta as estruturas de dados do sistema e como eles devem ser representados em um banco de dados.

Diante das atividades descritas anteriormente, podemos complementar a ideia de um projeto em que este retratará informações tanto da plataforma quando da especificação de requisitos e descrição de dados. O projeto em geral abrange também outros projetos menores e que, juntos, expõem todas as particularidades do software que está sendo desenvolvido.

Entre as décadas de 70 e 80 foram desenvolvidos alguns métodos, os quais antecederam a Linguagem de Modelagem Unificada (do inglês UML – *Unified Modeling Language*). Percebe-se que ao longo dos anos, o processo de desenvolvimento de um software passou por diversas fases. Obviamente, todas acompanharam a evolução dos próprios recursos computacionais, que foram exigindo funcionalidades cada vez mais complexas. A implementação de um sistema está relacionada à sua documentação e às definições estabelecidas entre a equipe e o cliente. Dessa forma, quanto mais detalhadas estiverem as informações, melhor, pois o risco de haver erros e má interpretação das

funcionalidades é reduzido. Uma forma de fazer isso é por meio do uso de diagramas, os quais mostram por meio gráfico todas as informações sobre um sistema.

Fique atento

O desenvolvimento do projeto de software deve ser elaborado a partir de uma detalhada Metodologia de Desenvolvimento de Sistemas e ancorado por uma competente NPTO (Normas e Padrões Técnicos Operacionais). Tais atividades envolvem muitas pessoas, as quais compõem a equipe multidisciplinar do projeto de software. Deve contemplar fases, subfases, produtos e pontos de avaliação, de acordo com o consenso dos envolvidos, sejam da própria organização, sejam prestadores de serviços. Essas atividades devem ser elaboradas dinamicamente, permitindo resgatar ou refazer subfases já elaboradas, a fim de contribuir com o sucesso do projeto, desde que não comprometa o seu resultado (REZENDE, 2005).

Diagramas

O projeto de um software, assim como sua documentação, traz os detalhes acerca das diversas partes de um software, seja arquitetural, de implementação e até de *design*, se for preciso. Um recurso que auxilia a equipe de desenvolvimento, principalmente na etapa de documentação e posteriormente de implementação do software, é o uso de diagramas, que são baseados na UML.

Para Fowler (2004), há três modos pelos quais as pessoas aplicam UML:

- UML como rascunho – diagramas incompletos e informais (frequentemente rascunhados à mão em quadros brancos) criados para explorar partes difíceis do problema ou espaço de soluções, explorando o poder das linguagens visuais.
- UML como planta de software – diagramas de projeto relativamente detalhados usados tanto para (1) a engenharia reversa para visualizar e melhor entender o código existente em diagramas UML quanto para (2) a geração de código (engenharia avante).
  - Em engenharia reversa, uma ferramenta UML lê o código-fonte ou o código binário e gera (tipicamente) diagramas UML de pacotes, de

classes e de sequência. Essas "plantas de software" podem ajudar o leitor a entender os elementos, a estrutura e as colaborações globais.
  - Antes da programação, alguns diagramas detalhados podem fornecer diretrizes para a geração de código (por exemplo, em Java), quer manualmente, quer automaticamente, com uma ferramenta. É comum que os diagramas sejam usados para uma parte do código e outra parte seja preenchida por um desenvolvedor que esteja codificando (talvez também aplicando rascunhos UML).
- UML como linguagem de programação – especificação executável completa de um sistema de software em UML. Código executável será automaticamente gerado, mas não é normalmente visto ou modificado por desenvolvedores; trabalha-se apenas na "linguagem de programação" UML. Esse uso da UML requer um modo prático de diagramar todo o comportamento ou a lógica (provavelmente usando diagramas de interação ou estado) e está ainda em desenvolvimento em termos de teoria, ferramentas robustas e usabilidade.

Já para Larman (2007), a UML descreve tipos de esboço de diagramas, tais como diagramas de classe e diagramas de sequência. Ela não superpõe a eles uma perspectiva de modelagem. Por exemplo, a mesma notação UML de diagrama de classes pode ser usada para desenhar imagens de conceitos do mundo real ou de classes de software em Java. Os diagramas podem ser vistos por três perspectivas:

- Perspectiva conceitual – os diagramas são interpretados como descrevendo coisas em uma situação do mundo real ou domínio de interesse.
- Perspectiva de especificação (software) – os diagramas (usando a mesma notação da perspectiva conceitual) descrevem abstrações de software ou componentes com especificações e interfaces, mas nenhum comprometimento com uma implementação particular (por exemplo, não especificamente uma classe em C# ou Java).
- Perspectiva de implementação (software) – os diagramas descrevem implementações de software em uma tecnologia particular (tal como Java).

Nota-se que as aplicações de diagramas UML descritos pelos autores Larman (2007) e Fowler (2004) se complementam e que a utilização dos diagramas serve para nortear toda a equipe de desenvolvimento. Um fato a ser acrescentado diante dessa informação é que os diagramas servem também para representar o sistema diante do cliente, fazendo com que a compreensão acerca de todas as funcionalidades e de suas aplicações seja expressa de forma gráfica.

Diferença entre diagramas da UML

Existem diversas formas de se aplicar os diversos diagramas existentes. Conforme Erickson e Siau (2007), cinco diagramas definidos pela UML são os mais utilizados para representar as particularidades de um sistema:

1. Diagrama de atividades, que mostra as atividades envolvidas em um processo ou no processamento de dados.
2. Diagrama de casos de uso, que mostra as interações entre um sistema e seu ambiente.
3. Diagrama de sequência, que mostra as interações entre os atores e o sistema e entre os componentes do sistema.
4. Diagrama de classe, que mostra as classes de objeto no sistema e as associações entre elas.
5. Diagrama de estado, que mostra como o sistema reage a eventos internos e externos.

A UML "[...] inclui diagramas de interação para ilustrar como os objetos interagem por meio de mensagens. Eles são usados para modelagem de objetos dinâmica. Há dois tipos comuns de diagramas de interação: diagramas de sequência e de comunicação" (LARMAN, 2007, p. 241). As Figuras 1 e 2 trazem demonstração de ambos os diagramas.

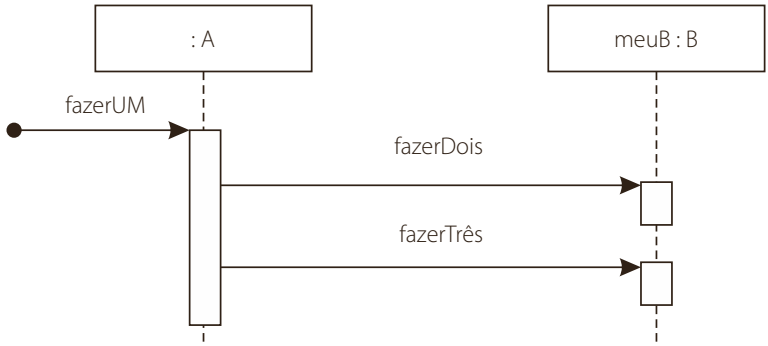

Figura 1. Exemplo de um diagrama de sequência.

Fonte: Larman (2007, p. 242).

Sob o ponto de vista de Larman (2007, p. 243), "[...] diagramas de sequência têm algumas vantagens sobre diagramas de comunicação. Talvez, antes de mais nada, a especificação UML seja mais centrada em diagramas de sequência – mais raciocínio e esforço têm sido colocados na notação e semântica. Assim, o apoio ferramental é melhor e mais opções de notação estão disponíveis. Também é mais fácil ver o fluxo de sequência de chamada com diagramas de sequência – simplesmente leia de cima para baixo. Com diagramas de comunicação, precisamos ler os números de sequência tais como '1:' e'2:'. Assim, diagramas de sequência são excelentes para documentação ou para ler facilmente uma sequência de fluxo de chamada obtida por engenharia reversa, gerada de código-fonte com uma ferramenta UML".

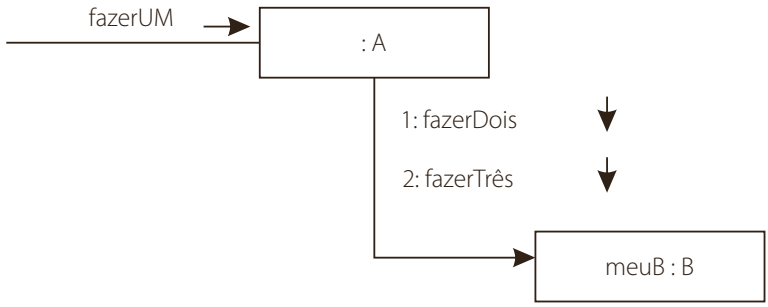

Figura 2. Exemplo de um diagrama de comunicação.

Fonte: Larman (2007, p. 242).

Porém, por outro lado, o autor complementa que os "[...] diagramas de comunicação têm vantagens quando se aplica 'UML como rascunho' para desenhar na parede (uma prática de modelagem ágil) porque eles são muito-mais eficientes em termos de espaço. Isso porque as caixas podem ser facilmente colocadas ou apagadas em qualquer lugar – horizontal ou vertical" (LARMAN, 2007, p. 243). Por fim, Larman (2007) traz um quadro comparativo entre os dois diagramas, apenas para sintetizar as informações sobre ambos (Quadro 1).

Quadro 1. Comparação entre os diagramas de sequência e de comunicação.

Tipo	Pontos fortes	Pontos fracos
Sequência	Mostra com clareza a sequência ou ordem temporal das mensagens. Amplo conjunto de opções detalhadas.	Deve ser estendido para a direita quando são acrescidos novos objetos; consome espaço na horizontal.
Comunicação	Economia de espaço – flexibilidade de adicionar novos objetos em duas dimensões.	É mais difícil ver a sequência das mensagens. Menos opções de notação.

Fonte: Larman (2007, p. 244).

Outro meio de se representar as funcionalidades de um sistema é por meio do diagrama de estado, que faz parte de uma modelagem dirigida a eventos. Sommerville (2011, p. 94) diz que "[...] é baseada na suposição de que um sistema tem um número finito de estados e que os eventos (estímulos) podem causar uma transição de estado para outro". Os diagramas de estado mostram os estados o sistema e os eventos que causam transições de um estado para os outros. Eles não mostram o fluxo de dados dentro do sistema, mas podem incluir informações adicionais sobre os processamentos realizados em cada estado, como mostra a Figura 3.

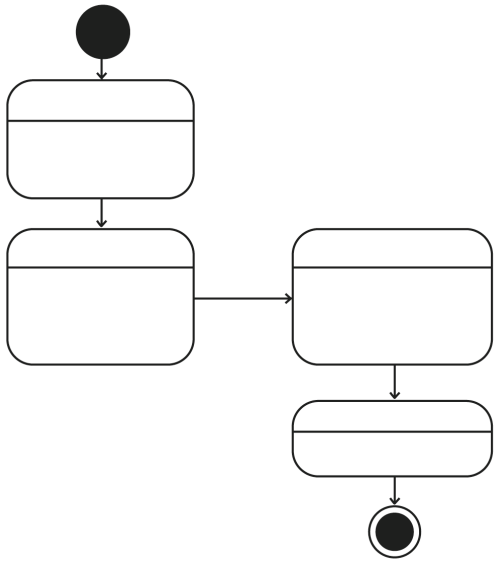

Figura 3. Diagrama de estado.

Sommerville (2011) complementa as informações sobre este tipo de diagrama quando diz que a em diagramas de estado da UML, retângulos arredondados representam os estados do sistema. Eles podem incluir uma breve descrição das ações tomadas nesse estado. As setas rotuladas representam estímulos que forçam uma transição de um estado para outro. Onde podem ser indicados os estados iniciais e finais usando círculos preenchidos.

O uso dos diagramas auxilia a todos na concepção do software, assim como em todas as ações praticadas neste processo de desenvolvimento de um sistema. O uso do diagrama fica a critério da necessidade do projeto e da equipe. Em um projeto, temos a oportunidade de utilizar todos, ou apenas alguns, diagramas, dentre as opções que a UML nos disponibiliza. Nota-se que eles trazem algumas notações em comum, o que muda, muitas vezes, são as informações que podem ser adicionadas ao diagrama com o intuito de enriquecê-lo.

Link

A Sthe Soares traz explicações acerca da concepção de um diagrama de classe por meio de uso de programas específicos para criação de diagramas:

https://goo.gl/KYhqck

Referências

ERICKSON, J.; SIAU, K. Theoretical and practical complexity of modeling methods. *Communications of the ACM*, New York, v. 50, n. 8, p. 46-51, Aug. 2007.

FOWLER, M. *UML distilled*. 3rd ed. Boston: Addison-Wesley, 2004.

LARMAN, C. *Utilizando UML e padrões*: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.

REZENDE, D. A. *Engenharia de software e sistemas de informação*. 3. ed. Rio de Janeiro: Brasport, 2005.

SOMMERVILLE, I. *Engenharia de software*. 9. ed. São Paulo: Pearson, 2011.

Leituras recomendadas

BRAUDE, E. J. *Projeto de software*: da programação à arquitetura: uma abordagem baseada em Java. Porto Alegre: Bookman, 2005.

HIRAMA, K. *Engenharia de software*: qualidade e produtividade com tecnologia. Rio de Janeiro: Elsevier, 2011.

HUMBLE, J.; FARLEY, D. *Entrega contínua*: como entregar software. Porto Alegre: Bookman, 2014.

PFLEEGER, S. L. *Engenharia de software*: teoria e prática. 2. ed.– São Paulo: Prentice--Hall, 2004.

PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SCHACH, S. R. *Engenharia de software*: os paradigmas clássicos e orientado a objetos. 7. ed. Porto Alegre: AMGH, 2010.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS